Progetto d'esame di Natural Language Processing

Agent-00Pharma: Un agente LangGraph per la classificazione delle interazioni farmaco-farmaco

---

Corso di laurea: LM informatica

Alex Rossi

Matricola: 0001242505

A.A. 2026/2027

Sommario

Agent-00Pharma è un agente basato su LangGraph per l'individuazione e la classificazione delle interazioni farmaco-farmaco in frasi di testi farmacologici, sviluppato e valutato su campioni del *DDI Corpus* (compito condiviso con la competizione *DDIExtraction 2013*).

Il sistema è composto da un modello linguistico "estrattore", assistito da un controllo di identità dei farmaci basato su *RxNorm*, e da un secondo modello "giudice" che verifica le predizioni e può restituire una critica all'estrattore, in un ciclo limitato.

Oltre all'implementazione, il lavoro valuta le diverse configurazioni con confronti appaiati sulle stesse coppie ed errori standard ottenuti con il metodo *bootstrap*.

L'ipotesi principale, secondo cui l'agente completo migliora il sistema di riferimento, non è confermata ($p = 0{,}18$ su 260 coppie); infatti un solo passaggio di verifica con un giudice prudente riduce i falsi positivi ma non raggiunge il criterio fissato in anticipo. Questo risultato viene ribadito anche dai test eseguiti su un secondo campione di conferma.

Indice

1 Introduzione **3**

1.1 Domande di ricerca 3

1.2 Contributi e struttura del report 4

2 Background tecnico **4**

2.1 Estrazione di DDI e il DDI Corpus 4

2.2 Convenzioni di annotazione 4

2.3 Campioni e rappresentatività statistica 5

2.4 Metriche di valutazione 6

2.5 Confronto con lo stato dell'arte 6

2.6 Stima riponderata sulla prevalenza reale 6

2.7 Confronto appaiato e test dei segni 7

2.8 RxNorm e normalizzazione dei nomi dei farmaci 8

3 Decisioni implementative **8**

3.1 Preparazione dati e input del sistema 8

3.2 Architettura del sistema 8

3.3 Modelli e infrastruttura 10

3.4 L'estrattore 10

3.5 Il giudice 10

3.6 Le configurazioni 11

3.7 Ottimizzazioni dell'infrastruttura 11

4 Protocollo sperimentale, risultati e analisi degli errori 12

4.1 Sviluppo e test 12

4.2 Metriche 13

4.3 Risultati sperimentali 13

4.3.1 Risultati *dev_sample* 14

4.3.2 Risultati sul test 14

4.3.3 Confronti appaiati e studio ablation 15

4.3.4 Stima alla prevalenza reale 16

4.3.5 Valutazione di conferma 16

4.4 Analisi degli errori 18

5 Conclusioni e lavori futuri 18

5.1 Risposte alle domande di ricerca 18

5.2 Limiti del progetto 19

5.3 Lavori futuri 19

Riferimenti bibliografici 21

1 Introduzione

L'estrazione automatica delle interazioni tra farmaci (*drug-drug interactions*, DDI) dai testi farmacologici, come i foglietti illustrativi o la letteratura biomedica, è un compito non banale per un sistema di intelligenza artificiale.

In questo lavoro consideriamo la versione più semplice del problema: data una frase in cui due farmaci sono già stati individuati, stabilire se questa afferma la presenza di un'interazione tra i due.

A tal fine, questo progetto segue il Task 9.2 di *DDIExtraction 2013* [10]: ogni coppia di farmaci riceve una tra cinque etichette:

- `none`: nessuna interazione;
- `effect`: presenza di interazione e descrizione della sua conseguenza clinica;
- `mechanism`: presenza di interazione e descrizione di come avviene;
- `advise`: presenza di interazione e raccomandazioni sull'uso dei farmaci;
- `int`: presenza di interazione senza ulteriori informazioni fornite.

Generalmente i modelli linguistici di grandi dimensioni (LLM) possono affrontare questa tipologia di compito, ma spesso commettono errori sistematici causati da:

- lettura di una frase senza riconoscere la *negazione dell'interazione* (ad esempio "nessuna interazione è stata osservata…");
- attribuzione alla coppia di farmaci del comportamento di un terzo farmaco (o di un'intera classe di farmaci);
- confusione tra i tipi di interazione, quando una frase ne descrive più di uno (ad esempio un effetto e una raccomandazione);
- presenza di nomi diversi per lo stesso farmaco (ad esempio nome commerciale e generico), che porta a trattare come interazione una coppia relativa a un solo principio attivo.

Per arginare questi errori si costruisce un sistema agentico con il framework *LangGraph* [5], seguendo l'idea che un agente possa migliorare le proprie risposte tramite un riscontro in linguaggio naturale [11, 6].

1.1 Domande di ricerca

Al fine di svolgere una valutazione quanto più oggettiva possibile rispetto al sistema sviluppato, le domande di ricerca sono le seguenti:

- **RQ1**: un agente con giudice e ciclo di critica migliora un singolo modello linguistico sul DDI Corpus?
- **RQ2**: che cosa riesce a verificare un giudice LLM? In particolare, quanto contano le istruzioni fornitegli al fine dell'analisi del Corpus?
- **RQ3**: come si valuta correttamente un sistema di questo tipo se si sfruttano campioni piccoli del Corpus?

1.2 Contributi e struttura del report

Il lavoro svolto si articola in tre direzioni: implementazione di un sistema agentico per il riconoscimento delle DDI, valutazione del sistema mediante metriche classiche (*precision, recall ed F1*) e analisi critica dei risultati con valutazione finale.

Dopo questa sezione introduttiva, il report è strutturato come segue:

- **Sezione 2 - Background tecnico**: in cui si forniscono maggiori dettagli riguardo il Corpus, i campioni usati, le metriche e le convenzioni attuate, oltre ad un approfondimento degli strumenti aggiuntivi che supportano il sistema sviluppato.
- **Sezione 3 - Decisioni implementative**: dove si descrivono l'architettura, i modelli e le configurazioni di sistema.
- **Sezione 4 - Protocollo sperimentale, risultati e analisi degli errori**: dove si analizzano, in modo critico, le metriche ottenute dalla valutazione del sistema.
- **Sezione 5 - Conclusioni e lavori futuri**: dove si risponde alle domande di ricerca e si propongono spunti per un possibile sviluppo futuro del progetto.

2 Background tecnico

In questa sezione si discutono le nozioni preliminari necessarie, e le convenzioni implementative attuate riguardo il trattamento dei dati, al fine di comprendere al meglio gli strumenti utilizzati nello sviluppo di questo progetto.

2.1 Estrazione di DDI e il DDI Corpus

Il *DDI Corpus* [4] è una raccolta di testi farmacologici in inglese dove sono presenti le sostanze annotate e le relative interazioni a coppie.

I dati sono distribuiti in forma di repository GitHub [9] e sono stati usati nel compito condiviso *DDIExtraction 2013* (SemEval-2013, Task 9) [10].

Le interazioni sono annotate a livello di frase ed hanno quattro tipi: `mechanism` (il modo in cui l'interazione avviene), `effect` (la sua conseguenza), `advise` (una raccomandazione) e `int` (un'interazione senza altre informazioni). Invece le coppie senza interazione sono definite tramite la classe `none`.

Il Corpus è costituito da due tipologie di fonti:

- **DrugBank**: frasi dalle schede dei farmaci, simili a foglietti illustrativi;
- **MedLine**: riassunti presi da articoli scientifici.

2.2 Convenzioni di annotazione

Le linee guida di annotazione [3] stabiliscono, per esempio, che le *classi di farmaci* contano come farmaci e che, in un'enumerazione, ogni elemento interagisce con il terzo farmaco e non con gli altri.

Nei dati si osserva inoltre che, quando lo stesso farmaco compare più volte nella frase, la relazione è annotata solo su alcune occorrenze: coppie con la stessa frase e gli stessi nomi possono quindi avere etichette diverse.

2.3 Campioni e rappresentatività statistica

Il Corpus è già suddiviso in un insieme di addestramento (*train*) e in un test ufficiale. Dal train, assegnando interi documenti, è stato ricavato un insieme di sviluppo (*dev*), così che frasi dello stesso documento non finiscano in insiemi diversi.

Nel progetto sviluppato, il train fornisce soltanto gli esempi inseriti nel prompt: non viene addestrato alcun modello. Inoltre, a causa dei limiti di richieste del piano gratuito dei modelli usati (500 richieste al giorno per modello), la valutazione del sistema si è basata non sull'intero Corpus ma su tre suoi campioni (Tabella 1): `dev_sample` (105 coppie), usato durante lo sviluppo del sistema; `test_sample` (260 coppie), usato una sola volta per la misura finale; e `test_confirm` (225 coppie, senza frasi in comune con il precedente), usato come test di conferma sulle metriche ottenute in `test_sample`.

Tabella 1: Composizione dei campioni rispetto agli insiemi completi del Corpus (numero di coppie per etichetta).

Insieme	Coppie	Doc.	`none`	`effect`	`mech.`	`advise`	`int`
train	25.719	562	22.132	1.491	1.159	754	183
dev	2.072	62	1.639	196	160	72	5
test (ufficiale)	5.716	191	4.737	360	302	221	96
`dev_sample`	105	40	40	20	20	20	5
`test_sample`	260	117	100	40	40	40	40
`test_confirm`	225	108	100	40	40	40	5

La scelta di usare campioni del Corpus, invece che la sua interezza, fa sorgere due problemi distinti:

- **Composizione**: i campioni sono stratificati per classe, mentre nel Corpus circa l'83% delle coppie non presenta interazione. Di conseguenza i *falsi positivi* pesano meno che nella realtà e le metriche non sono confrontabili con quelle del compito condiviso [10]. Per questo si affianca una stima *riponderata*, in cui ogni coppia pesa in base alla frequenza reale della sua classe (Sezione 2.6).
- **Dimensione**: con poche centinaia di coppie ogni metrica è una stima affetta da incertezza. La si quantifica con l'*errore standard* e con l'*intervallo di confidenza al 95%*, ottenuti con il metodo *bootstrap a cluster*: si effettua un *ricampionamento con reimmissione* delle frasi del campione (con le rispettive coppie, perché coppie della stessa frase non sono indipendenti) e si ricalcola la metrica; nel nostro caso il processo è ripetuto 2000 volte.

  L'*errore standard* è la deviazione standard delle stime ottenute e l'intervallo usato ne contiene il 95% centrale. Se l'intervallo di una differenza tra due sistemi contiene lo zero, non si può escludere che l'effetto sia nullo (Sezione 2.7).

2.4 Metriche di valutazione

La qualità del sistema è stata misurata con le metriche *precision, recall ed F1*, come indicato nel compito condiviso [10]:

$$P = \frac{TP}{TP + FP} \qquad R = \frac{TP}{TP + FN} \qquad F_1 = \frac{2PR}{P + R}$$

dove *TP*, *FP* e *FN* sono rispettivamente i *True Positive* (coppie con interazione correttamente riconosciute), i *False Positive* (coppie senza interazione scambiate per interazioni) e i *False Negative* (coppie con interazione scambiate per coppie senza interazione).

Si distinguono due livelli di valutazione:

- **Detection**: si valuta soltanto se la coppia interagisce, a prescindere dal tipo;
- **Detection e classificazione**: una coppia è corretta solo se sono giusti sia la presenza dell'interazione, sia il suo tipo. Nello specifico, si riportano la *F1 micro* (i conteggi di tutte le classi di interazione vengono sommati e, solo successivamente, si calcola la relativa *F1*) e la *F1 macro* (media delle *F1* delle singole classi).

  La classe `none` è esclusa da questo conteggio e quindi anche dalle relative medie.

Si affiancano inoltre due conteggi descrittivi: i *False Positive* ('FP') e le *interazioni perse* ('perse'), cioè le coppie con interazione classificate invece come `none`.

2.5 Confronto con lo stato dell'arte

Nel compito di estrazione delle interazioni, in cui le sostanze sono fornite, il miglior sistema ha ottenuto una *F1* del 65,1% nella *detection e classificazione* e dell'80% nella sola *detection* [10]. Poiché, come detto, il sistema sviluppato è stato valutato solo su *campioni* del Corpus originale, il confronto diretto con questi risultati non è quindi possibile.

2.6 Stima riponderata sulla prevalenza reale

La *precision* dipende dal rapporto tra veri e falsi positivi, e quindi dalla proporzione di coppie senza interazione presenti nei dati. Nei campioni questa proporzione è del 38–44%, mentre nel Corpus completo è circa l'83% (Tabella 1): lo stesso sistema avrebbe quindi, sui dati reali, una precision molto più bassa di quella misurata sul campione.

Assumendo che i tassi di errore osservati nel campione valgano anche sull'insieme completo, per stimare le metriche alla prevalenza reale si assegna a ogni coppia i di classe c il peso

$$w_i = \frac{N_c}{n_c},$$

dove N_c è il numero di coppie della classe c nell'insieme completo da cui il campione è tratto (il *test ufficiale* per `test_sample` e `test_confirm`, mentre il *dev* per `dev_sample`) e n_c è il numero di coppie della stessa classe nel campione. In questo modo ogni coppia del campione "rappresenta" w_i coppie dell'insieme completo.

Per `test_sample` i pesi sono circa 47,4 per `none` (4.737/100), 9,0 per `effect` (360/40), 7,6 per `mechanism` (302/40), 5,5 per `advise` (221/40) e 2,4 per `int` (96/40).

Questo calcolo è una stima approssimata: ogni coppia senza interazione del campione pesa circa 47 volte, quindi il risultato dipende molto dalle sole 100 coppie senza interazione (inoltre l'analisi effettuata è esplorativa, ovvero introdotta a seguito dell'osservazione dei risultati delle prime valutazioni).

Nel *bootstrap* i pesi si ricalcolano a ogni ricampionamento, perché la composizione per classe del campione ricampionato cambia:

$$P' = \frac{\sum_{i \in TP} w_i}{\sum_{i \in TP} w_i + \sum_{i \in FP} w_i}, \qquad R' = \frac{\sum_{i \in TP} w_i}{\sum_{i \in TP} w_i + \sum_{i \in FN} w_i},$$

da cui si ricavano le *F1* riponderate. Nello specifico, le metriche di *precision* e *recall* si ricalcolano sostituendo i conteggi con somme dei pesi.

In generale, in questo lavoro, si sono quindi effettuate analisi decise prima di osservare i risultati (*confermative*)e dopo averli osservati (*esplorative*, e hanno valore di indizio più che di prova).

2.7 Confronto appaiato e test dei segni

Per stabilire se un sistema B è migliore di un sistema A non basta confrontare due valori di *F1*, che sono stime ottenute su pochi dati.

Il **confronto appaiato** valuta i due sistemi sulle *stesse* coppie e osserva, per ciascuna, se A e B hanno ragione. Le coppie su cui sono entrambi giusti, o entrambi sbagliati, non dicono nulla sulla differenza: contano solo le coppie *discordanti*.

Siano:

- b: coppie che B classifica correttamente e A no (B *ripara*);
- c: coppie che A classifica correttamente e B no (B *rompe*).

Se i due sistemi fossero equivalenti, ogni coppia discordante andrebbe a favore di B o di A con uguale probabilità. Con $n = b + c$ coppie discordanti e $k = \min(b, c)$, il *test dei segni esatto* calcola il *p-value* a due code:

$$p = \min\left(1,\ 2\sum_{i=0}^{k} \binom{n}{i} \frac{1}{2^n}\right).$$

Il **p-value** è la probabilità, se i sistemi fossero equivalenti, di osservare uno squilibrio almeno così grande: un valore piccolo indica che lo squilibrio è poco compatibile con il caso, mentre un valore alto non significa che i sistemi siano uguali, ma che i dati non bastano per dirlo.

Poiché le coppie della stessa frase non sono indipendenti, lo stesso conteggio si ripete anche a livello di *frase*: per ogni frase si calcola il saldo tra coppie *riparate–rotte* e si applica il test al numero di frasi con saldo positivo contro quelle con saldo negativo.

Per misurare la differenza tra due metriche (per esempio la *F1* di *detection* riponderata), si usa il **bootstrap appaiato**: i ricampionamenti delle frasi sono gli stessi per i due sistemi, a ogni ricampionamento si calcola la differenza tra le metriche e se ne ricavano l'errore standard e l'intervallo di confidenza al 95%.

Quando si eseguono molti confronti, la probabilità di ottenere per caso almeno un risultato "significativo" cresce (*confronti multipli*), per cui si riportano tutti i confronti eseguiti.

In generale, l'uso di test di significatività adeguati è fondamentale (come indicato in [1]).

2.8 RxNorm e normalizzazione dei nomi dei farmaci

RxNorm è una terminologia normalizzata dei farmaci prodotta dalla *U.S. National Library of Medicine* (NLM): associa a ciascun nome un identificativo e ne collega le relazioni, integrando nomi e relazioni provenienti da numerosi vocabolari di partenza [7]. Strumenti come RxNav ne permettono l'interrogazione, anche tramite un'API pubblica [12] (utilizzata all'interno di questo progetto).

RxNorm *non* contiene informazioni sulle interazioni, bensì serve a stabilire a che cosa si riferisce un nome. Infatti uno stesso farmaco può comparire con nomi diversi: il nome del principio attivo (generico), nomi commerciali, sali oppure tramite abbreviazioni.

3 Decisioni implementative

In questa sezione si discute l'architettura e gli strumenti usati all'interno del progetto.

3.1 Preparazione dati e input del sistema

Prima di effettuare una corretta valutazione delle coppie, si deve preparare il Corpus trasformandolo in un record per coppia.

Usando la versione XML del Corpus [9] (distribuita dagli autori con licenza CC BY-NC), ogni documento contiene frasi, entità (con testo, tipo e posizione) e coppie di entità con la relativa etichetta (il *gold*, definito nelle coppie del *train set* e del *test set*).

L'**input al sistema** è quindi una frase con due menzioni di farmaci, D_1 e D_2, marcate per posizione con `<D1>...</D1>` e `<D2>...</D2>`. In questo modo, se un farmaco compare più volte nella frase, è chiaro a quale occorrenza si riferisce la coppia.

3.2 Architettura del sistema

L'architettura del sistema è riassunta nella Figura 1.

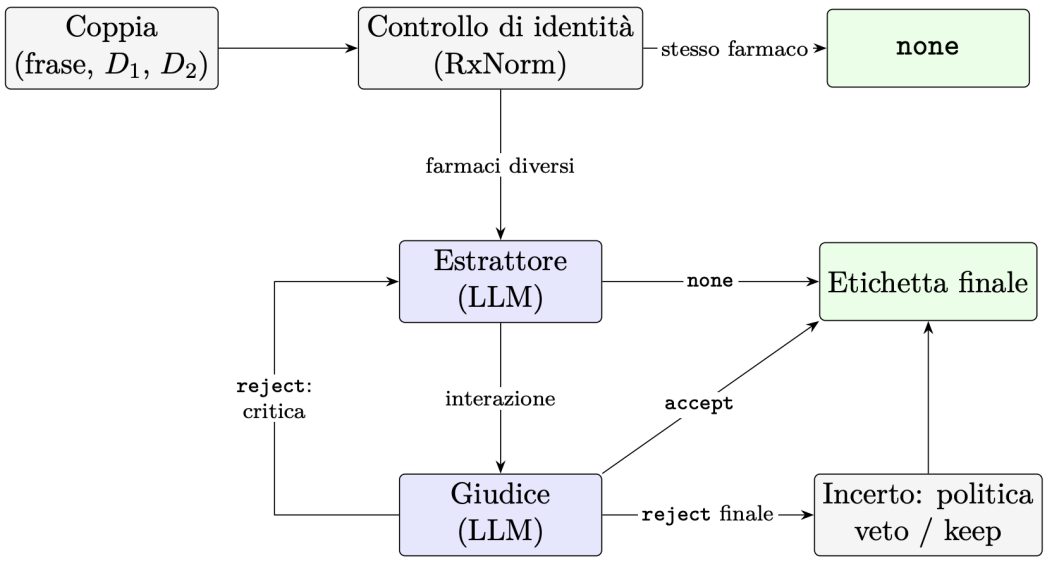


Figura 1: Astrazione dell'architettura del sistema (*Nota*: la freccia `none` vale per la variante *solo interazioni*).

Seguendo la struttura del grafo, il sistema riceve in input una frase (elaborata secondo quanto descritto nella sezione 3.1) e la fa passare attraverso i seguenti nodi:

1. Il nodo del **controllo di identità**: confronta i nomi di D_1 e D_2 e stabilisce se indicano lo stesso farmaco. Nello specifico, due nomi sono considerati lo stesso farmaco se coincidono oppure se tramite l'API di RxNorm vengono ricondotti allo stesso insieme di principi attivi.

   Se risultano essere lo stesso farmaco, il sistema produce in output l'etichetta `none` senza interrogare alcun modello (in quanto un farmaco non può interagire con sé stesso), altrimenti passa al nodo successivo.

2. Il nodo **estrattore**: tramite un *Large Language Model* (LLM), riconosce se vi è un'interazione e propone un'etichetta tra le cinque classificazioni disponibili nel sistema.

   Se il sistema è usato nella configurazione con variante *solo interazioni*, se l'etichetta proposta è `none` diventa subito l'etichetta finale; se è un'interazione passa al giudice. Se invece si usa nella variante *anche none*, l'etichetta proposta passa sempre alla valutazione del giudice.

3. Il nodo **giudice**: riceve la frase, e l'etichetta proposta, e risponde con un verdetto (`accept` o `reject`), un motivo e una critica di una o due frasi (se valuta che l'etichetta proposta è errata).

   Se accetta, l'etichetta proposta diventa l'output del sistema. Se rifiuta, la critica viene restituita all'estrattore, che riprova la valutazione.

   Il processo si ripete fino a un massimo di tre tentativi complessivi. Se al termine il giudice rifiuta ancora, la coppia passa al nodo successivo con l'ultima etichetta proposta.

4. Il nodo **incerto**: decide l'etichetta finale in base alla *politica* scelta. Con *politica keep* si produce in output l'ultima etichetta proposta dall'estrattore; con *politica veto* si scarta l'etichetta proposta e si produce in output `none`.

Infine l'output, in formato JSON, viene elaborato via parsing per estrarre l'etichetta stabilita dal sistema (quindi una tra `none, effect, mechanism, advise` e `int`).

3.3 Modelli e infrastruttura

In questo progetto, per quanto riguarda gli LLM, si sfruttano modelli di Google Gemini con piano gratuito a livello di costi. Nello specifico si usa `gemini-3.5-flash-lite` come estrattore e `gemini-3.1-flash-lite` come giudice.

L'uso di due modelli distinti risponde a due ragioni: separare i ruoli (chi propone l'etichetta non è la stessa istanza che la verifica) e disporre di quote giornaliere separate per l'uso delle chiamate API di ciascun modello (circa 500 richieste al giorno con un massimo di 250000 token giornalieri [2]).

Poiché entrambi appartengono alla stessa famiglia, resta comunque il rischio che il giudice tenda a confermare le risposte dell'estrattore: i modelli linguistici usati come valutatori tendono infatti a preferire i propri output [8] e mostrano *bias di auto-valorizzazione* [13].

3.4 L'estrattore

L'estrattore è il modello `gemini-3.5-flash-lite`. Per ogni coppia riceve un prompt, scritto in inglese, che contiene: le definizioni delle cinque etichette, la frase con le due menzioni marcate, l'indicazione di giudicare solo quelle due menzioni (e solo ciò che la frase afferma) e dieci esempi, due per etichetta, scelti in modo deterministico tra le frasi più brevi del *train* (*few-shot prompting*).

L'estrattore risponde quindi in formato JSON con l'etichetta e questa viene letta da un *parser*: una risposta che risulta illeggibile viene contata come *errore*.

Il primo tentativo dell'agente usa esattamente il prompt della *baseline*, per cui ogni differenza tra le configurazioni dipende dai componenti aggiunti e non dal prompt. Dopo un rifiuto del giudice, il prompt della *ripetizione* include anche la critica e la richiesta *"mantenere la risposta se si è sicuri"*.

3.5 Il giudice

Il giudice riceve la frase, e l'etichetta proposta, e risponde in formato JSON con tre campi: un verdetto (`accept` o `reject`), un motivo e una critica di una o due frasi (restituita all'estrattore nelle configurazioni con ciclo). Il motivo vale `none` se il giudice accetta; se rifiuta, indica che cosa è errato:

- `not_between_pair`: la frase non afferma un'interazione tra *proprio quelle due* entità (per esempio perché sono solo elencate insieme);
- `no_interaction_stated`: la frase non afferma alcuna interazione (per esempio perché la nega o descrive uno studio);

- `wrong_type`: l'interazione c'è, ma l'etichetta proposta è errata.

In questo progetto sono state scritte due *rubriche*, cioè gli insiemi di criteri (inseriti nel prompt) che il giudice applica:

- **Rubrica v1**: scritta a partire da alcuni errori osservati sul *dev*; controlla se la frase afferma un'interazione tra proprio quelle due entità (con esclusioni: elenchi, due farmaci che interagiscono entrambi con un terzo, presenza di classe e membro, presenza di confronti nella frase oppure descrizioni di studi) e se l'etichetta proposta è corretta; rifiuta se uno dei due controlli fallisce.
- **Rubrica v2**: scritta dopo aver letto le convenzioni del Corpus [3], in quanto la v1 le contraddiceva in alcuni casi (per esempio escludeva le classi). Enuncia che: le classi contano come farmaci, in un'enumerazione ogni elemento interagisce con il terzo e non con gli altri, classe e membro non interagiscono, una classe in apposizione a un farmaco è solo una descrizione e un titolo *"Interazione di X e Y"* sostiene `int`. Inoltre si esplicita che serve una motivazione (tra quelle elencate precedentemente) per rifiutare, e vale la regola *"rifiuta solo se sei sicuro, nel dubbio accetta"*.

3.6 Le configurazioni

Le configurazioni sono cumulative: ciascuna aggiunge un componente alla precedente.

- **baseline**: una sola chiamata all'estrattore;
- **identity**: baseline più controllo di identità;
- **judge**: identity più un giudice che verifica le interazioni proposte, con un solo tentativo: se rifiuta, l'etichetta diventa `none` (veto);
- **agent**: judge con il ciclo di critica, fino a tre tentativi dell'estrattore, e la politica (*veto* o *keep*) per le coppie rimaste incerte.

Il nome include la rubrica e le opzioni: `judge` e `agent` usano la rubrica v1, mentre il suffisso `v2` indica la rubrica v2. Per l'agente, `3` è il numero massimo di tentativi e `all` indica che il giudice verifica anche i `none` (senza `all` verifica solo le interazioni).

Per esempio, `judgev2` è il veto con rubrica v2, mentre `agent3allv2` è l'agente con tre tentativi, giudice anche sui `none` e rubrica v2.

3.7 Ottimizzazioni dell'infrastruttura

Con un piano gratuito, quote giornaliere limitate ed errori temporanei dei servizi, per rendere gli esperimenti completabili e ripetibili sono stati introdotti alcuni accorgimenti.

- **Cache su disco**: ogni risposta dei modelli e ogni richiesta a RxNorm vengono salvate in locale. Una richiesta ripetuta non consuma quota e restituisce la stessa risposta, e le configurazioni che condividono le stesse chiamate le riusano.
- **Limitatore di velocità**: tra due chiamate allo stesso modello si rispetta un intervallo di tempo minimo, in modo da non superare le richieste al minuto consentite.

- **Run ripresa dal punto in cui interrotta**: ogni predizione viene scritta su file appena prodotta, per cui una run interrotta riprende, rilanciando lo stesso comando, dalle coppie mancanti.
- **Tracciabilità pensiero agente**: di ogni coppia si conserva il percorso completo dell'agente (etichette proposte, verdetti, critiche, esito), così da svolgere analisi a posteriori (esplorative), come confrontare le politiche *veto* e *keep*, senza nuove chiamate.
- **Tag di versione e semi**: i tag del repository (`freeze-pre-test`, `pre-conferma`) fissano il codice usato per i risultati; i sorteggi dei campioni usano semi fissati (42, e 2026 per il campione di conferma).

I modelli non sono deterministici (l'estrattore ignora la temperatura), quindi la ripetibilità delle singole risposte è garantita dalla cache e non dalla generazione stessa.

4 Protocollo sperimentale, risultati e analisi degli errori

In questa sezione si discute il protocollo sperimentale, si analizzano le metriche relative alle prestazioni ottenute dal sistema nella classificazione delle coppie e si confrontano i risultati ottenuti dalle configurazioni valutate.

4.1 Sviluppo e test

Come già descritto nella Sezione 2.3, la valutazione si basa su tre campioni: il `dev_sample` (105 coppie), usato per lo sviluppo di prompt, rubriche e politiche; il `test_sample` (260 coppie), usato *una sola volta*, alla fine, per ottenere le metriche finali; e il `test_confirm` (225 coppie, senza frasi in comune con il `test_sample`), usato per verificare su dati nuovi il risultato emerso dal primo test. Prima del test il codice è stato congelato (con un tag di versione nel repository) e sono state scritte le configurazioni da valutare, le metriche e l'ipotesi principale.

L'obiettivo del protocollo è verificare l'**ipotesi principale**, secondo cui la configurazione `agent3allv2` (sistema agentico con fino a tre tentativi dell'estrattore, giudice che valuta tutte le etichette proposte, compresi i `none`, rubrica *v2* e politica *keep*) ottiene prestazioni migliori della configurazione `identity`, cioè un *micro F1* più alto sul campione bilanciato, misurato con un confronto appaiato e il test dei segni esatto.

Si precisa che l'ipotesi principale, le configurazioni e le metriche sono state fissate prima di osservare il test, mentre le analisi aggiuntive (la stima riponderata, la scelta di `judgev2` come configurazione migliore e le altre configurazioni eseguite su `test_confirm`) sono state decise dopo aver osservato i risultati. Inoltre, le scelte sull'ambito del giudice e sulla politica *keep* sono state fatte guardando il *dev*, i cui risultati sono ottimistici.

4.2 Metriche

Come già descritto nella Sezione 2.4, per la valutazione si utilizzano *precision*, *recall* ed *F1*, seguendo lo schema di valutazione del compito condiviso. In particolare:

- la classe `none` non entra nelle medie, in quanto è di gran lunga la più frequente e gonfierebbe i punteggi;
- la *macro F1* è la media delle *F1* delle quattro classi positive; il compito condiviso descrive la media macro come media di precision e recall per tipo, quindi la definizione numerica può differire leggermente;
- si riportano anche due conteggi descrittivi (non presenti nel compito condiviso ma utili per la comprensione): i *falsi positivi* su `none` (`FP`) e le *interazioni vere cancellate* (`perse`), cioè le coppie con interazione classificate come `none`;
- un'etichetta non valida è trattata come `none` e contata a parte.

Poiché i campioni sono piccoli, una differenza di *F1* tra due configurazioni può dipendere da poche coppie. Si confrontano quindi i due sistemi sulle *stesse* coppie: si contano quelle in cui la seconda configurazione è corretta e la prima errata (coppie "*riparate*") e viceversa (coppie "*rotte*"); poi si applica il test dei segni esatto (Sezione 2.7), anche a livello di frase.

L'incertezza delle metriche è quantificata con l'errore standard e con l'intervallo di confidenza al 95%, ottenuti con il bootstrap per frasi.

Accanto alle metriche sul campione si riporta una stima *riponderata* sulla prevalenza reale (Sezione 2.6): nel Corpus l'83% delle coppie è `none`, mentre nei campioni la quota è del 38–44%, quindi i falsi positivi pesano meno sul campione che nella realtà. La stima assume che i tassi di errore osservati nel campione valgano sull'insieme completo e non rende i risultati confrontabili con quelli del compito condiviso. Questa è un'analisi esplorativa, introdotta dopo aver osservato i primi risultati.

4.3 Risultati sperimentali

Di seguito si presentano i risultati delle metriche su tutte le configurazioni valutate (nell'ordine in cui sono state ottenute): sul *dev*, sul *test*, confronti appaiati e ablation, stima alla prevalenza reale e conferma su un secondo campione.

In tutte le tabelle presentate valgono le seguenti regole:

- i valori sono *stima* ± *errore standard*, cioè la deviazione standard di 2000 stime ottenute ricampionando le frasi (bootstrap);
- `FP` sono i falsi positivi su `none`;
- `persi` le interazioni vere classificate `none` (conteggi);
- "*det.*" indica la detection, con `P` e `R` senza errore standard;
- viene riportato in grassetto il *micro F1* più alto.
- nei nomi delle configurazioni, *v1*/*v2* indica la rubrica del giudice, *solo int.*/*anche none* l'ambito del giudice (solo interazioni, oppure anche i `none`) e *veto*/*keep* la

politica per le coppie incerte; "judge" non ha il ciclo di critica, "agente" sì; la configurazione "*agente v1, solo int.*" è valutata solo sul *dev*.

4.3.1 Risultati *dev_sample*

Tabella 2: `dev_sample`: 105 coppie (40 senza interazione, 65 con interazione), 80 frasi.

Configurazione	micro F1	macro F1	det. F1	det. P / R	FP	persi
baseline	$0{,}829 \pm 0{,}036$	$0{,}813 \pm 0{,}083$	$0{,}886 \pm 0{,}026$	0,827 / 0,954	13	3
identity	$0{,}835 \pm 0{,}036$	$0{,}818 \pm 0{,}083$	$0{,}892 \pm 0{,}026$	0,838 / 0,954	12	3
judge v1 (veto)	$0{,}744 \pm 0{,}043$	$0{,}657 \pm 0{,}066$	$0{,}777 \pm 0{,}039$	0,839 / 0,723	9	18
agente v1, solo int., veto	$0{,}748 \pm 0{,}042$	$0{,}711 \pm 0{,}071$	$0{,}780 \pm 0{,}038$	0,828 / 0,738	10	17
agente v1, solo int., keep	$0{,}800 \pm 0{,}040$	$0{,}751 \pm 0{,}070$	$0{,}846 \pm 0{,}033$	0,846 / 0,846	10	10
agente v1, anche none, veto	$0{,}768 \pm 0{,}044$	$0{,}726 \pm 0{,}072$	$0{,}800 \pm 0{,}040$	0,833 / 0,769	10	15
agente v1, anche none, keep	$0{,}855 \pm 0{,}036$	$0{,}862 \pm 0{,}064$	$0{,}899 \pm 0{,}028$	0,849 / 0,954	11	3
judge v2 (veto)	$0{,}836 \pm 0{,}036$	$0{,}819 \pm 0{,}082$	$0{,}881 \pm 0{,}028$	0,855 / 0,908	10	6
agente v2, anche none, veto	$0{,}849 \pm 0{,}036$	$0{,}857 \pm 0{,}063$	$0{,}892 \pm 0{,}028$	0,838 / 0,954	12	3
agente v2, anche none, keep	$\mathbf{0{,}865 \pm 0{,}034}$	$\mathbf{0{,}870 \pm 0{,}063}$	$\mathbf{0{,}908 \pm 0{,}026}$	0,842 / 0,985	12	1

La baseline ha recall alta (0,954) ma 13 falsi positivi su 40; il controllo di identità corregge una sola coppia.

Il giudice v1 come veto peggiora il sistema (micro F1 0,744: toglie 3 falsi positivi ma cancella 15 interazioni vere in più).

Con il ciclo la politica *keep* è migliore del veto, e solo la variante con il giudice anche sui `none` e *keep* non peggiora `identity` (0,855).

Con la rubrica v2 il solo veto è neutro (0,836) e l'agente con *keep* arriva a 0,865.

In generale, questi valori sono da considerare ottimistici. Inoltre, per quanto riguarda l'*ambito del giudice*, *keep* e rubrica v2 sono stati scelti, e sistemati, su questo campione (errore standard del micro F1: 0,034–0,044).

4.3.2 Risultati sul test

Tabella 3: `test_sample`: 260 coppie (100 senza interazione, 160 con interazione), 169 frasi.

Configurazione	micro F1	macro F1	det. F1	det. P / R	FP	persi
baseline	$0{,}754 \pm 0{,}059$	$0{,}751 \pm 0{,}061$	$0{,}904 \pm 0{,}020$	0,868 / 0,944	23	9
identity	$0{,}757 \pm 0{,}059$	$0{,}752 \pm 0{,}061$	$0{,}907 \pm 0{,}020$	0,873 / 0,944	22	9
judge v1 (veto)	$0{,}743 \pm 0{,}062$	$0{,}730 \pm 0{,}063$	$0{,}868 \pm 0{,}027$	0,917 / 0,825	12	28
agente v1, anche none, veto	$0{,}731 \pm 0{,}061$	$0{,}717 \pm 0{,}064$	$0{,}867 \pm 0{,}027$	0,899 / 0,838	15	26
agente v1, anche none, keep	$0{,}758 \pm 0{,}060$	$0{,}750 \pm 0{,}063$	$0{,}911 \pm 0{,}020$	0,892 / 0,931	18	11
judge v2 (veto)	$\mathbf{0{,}773 \pm 0{,}060}$	$\mathbf{0{,}765 \pm 0{,}060}$	$\mathbf{0{,}922 \pm 0{,}018}$	0,919 / 0,925	13	12
agente v2, anche none, veto	$0{,}771 \pm 0{,}060$	$0{,}764 \pm 0{,}061$	$0{,}924 \pm 0{,}018$	0,904 / 0,944	16	9
agente v2, anche none, keep	$0{,}770 \pm 0{,}059$	$0{,}762 \pm 0{,}060$	$0{,}927 \pm 0{,}017$	0,900 / 0,956	17	7

Le differenze sono piccole: il micro F1 va da 0,731 a 0,773 con errore standard di circa 0,06, quindi tutti i valori stanno entro un errore standard.

Cambia il compromesso: il giudice v1 come veto riduce i falsi positivi da 22 a 12 ma perde 28 interazioni (9 per `identity`); il giudice v2 ne toglie 9 e ne perde 3 in più. L'errore principale riguarda `int`: nella baseline 19 coppie su 40 sono predette `effect` (recall 0,45), e nessuna variante le recupera (Sezione 4.4).

4.3.3 Confronti appaiati e studio ablation

Tabella 4: Confronti appaiati con `identity` su `test_sample` (etichetta esatta).

B	giuste (id. → B)	ripara / rompe	p	frasi
baseline → identity	203 → 204	1 / 0	1,000	1 / 0
agente v1, anche none, keep	204 → 206	7 / 5	0,774	7 / 5
agente v2, anche none, keep	204 → 210	10 / 4	0,180	10 / 4
agente v2, anche none, veto	204 → 210	11 / 5	0,210	11 / 5
judge v2 (veto)	204 → 211	9 / 2	0,065	9 / 2

L'ipotesi principale non è confermata: l'agente completo (`agent3allv2`, *keep*) ripara 10 coppie e ne rompe 4 ($p = 0{,}18$), e nessun confronto scende sotto 0,05.

Il confronto diretto tra `judgev2` e l'agente completo non mostra differenze (211 contro 210, $p = 1$), ma l'agente costa di più (280 contro 173 chiamate al giudice).

Le 9 coppie riparate da `judgev2` hanno tutte etichetta `none`: cinque sono risultati negati letti come interazioni, due sono coppie tra elementi di un elenco, due sono coppie classe-membro (Sezione 4.4).

La Tabella 5 isola il contributo di ciascun componente e permette di analizzare meglio il confronto tra le prestazioni delle configurazioni.

Tabella 5: Studio ablation: ΔF1 ± errore standard (valori non arrotondati); ΔFP e Δpersi sono conteggi.

	`test_sample`			`test_confirm`		
Componente (confronto)	ΔF1	ΔFP	Δpersi	ΔF1	ΔFP	Δpersi
Controllo di identità (baseline → identity)	$+0{,}002 \pm 0{,}002$	−1	0	$+0{,}008 \pm 0{,}005$	−3	0
Giudice v2, veto (identity → judge v2)	$+0{,}016 \pm 0{,}010$	−9	+3	$+0{,}008 \pm 0{,}009$	−5	+3
Ciclo e controllo dei `none` (judge v2 → agente v2, veto)	$-0{,}002 \pm 0{,}007$	+3	−3	$-0{,}006 \pm 0{,}012$	+7	−3
Politica *keep* (agente v2: veto → keep)	$-0{,}001 \pm 0{,}005$	+1	−2	$+0{,}006 \pm 0{,}007$	+1	−2
Rubrica v2 invece di v1 (judge v1 → judge v2)	$+0{,}029 \pm 0{,}018$	+1	−16	$+0{,}005 \pm 0{,}014$	+6	−8

Il solo veto del giudice v2 è il componente che sposta di più il compromesso (9 e 5 falsi positivi in meno, ma 3 interazioni perse); il ciclo con il controllo dei `none` non riduce i

falsi positivi (+3 e +7), la politica *keep* e il controllo di identità hanno effetti minimi, e la rubrica v2 perde molte meno interazioni della v1 (16 e 8 in meno).

In nessun passo ΔF1 supera due errori standard.

4.3.4 Stima alla prevalenza reale

Poiché nei campioni la quota di coppie senza interazione è molto inferiore a quella del Corpus, si riportano le stime riponderate (si effettua un'analisi esplorativa secondo quanto detto nella sezione 2.6).

Tabella 6: Stima riponderata alla prevalenza reale su `test_sample`: stima ± errore standard.

Sistema	detection P	detection R	detection F1	micro F1
baseline	$0{,}457 \pm 0{,}052$	$0{,}936 \pm 0{,}029$	$0{,}614 \pm 0{,}047$	$0{,}553 \pm 0{,}046$
identity	$0{,}468 \pm 0{,}053$	$0{,}936 \pm 0{,}029$	$0{,}624 \pm 0{,}048$	$0{,}561 \pm 0{,}046$
judge v1 (veto)	$0{,}587 \pm 0{,}068$	$0{,}824 \pm 0{,}041$	$0{,}685 \pm 0{,}050$	$0{,}636 \pm 0{,}049$
judge v2 (veto)	$0{,}595 \pm 0{,}069$	$0{,}925 \pm 0{,}029$	$0{,}724 \pm 0{,}052$	$0{,}653 \pm 0{,}050$
agente v1, anche none, keep	$0{,}514 \pm 0{,}060$	$0{,}920 \pm 0{,}032$	$0{,}659 \pm 0{,}050$	$0{,}596 \pm 0{,}048$
agente v2, anche none, keep	$0{,}534 \pm 0{,}064$	$0{,}944 \pm 0{,}028$	$0{,}682 \pm 0{,}051$	$0{,}613 \pm 0{,}050$

Tabella 7: Differenza rispetto a `identity` alla prevalenza reale, riportati nella forma: *differenza ± errore standard [Intervallo di Confidenza al 95%]* (dal 2,5° al 97,5° percentile delle 2000 stime).

Sistema	Δ detection F1	Δ micro F1
baseline	$-0{,}010 \pm 0{,}010$ $[-0{,}035;\ +0{,}000]$	$-0{,}009 \pm 0{,}009$ $[-0{,}031;\ +0{,}000]$
judge v1 (veto)	$+0{,}062 \pm 0{,}038$ $[-0{,}012;\ +0{,}139]$	$+0{,}074 \pm 0{,}033$ $[+0{,}014;\ +0{,}141]$
judge v2 (veto)	$+0{,}101 \pm 0{,}034$ $[+0{,}040;\ +0{,}174]$	$+0{,}092 \pm 0{,}030$ $[+0{,}037;\ +0{,}158]$
agente v1, anche none, keep	$+0{,}036 \pm 0{,}035$ $[-0{,}031;\ +0{,}108]$	$+0{,}035 \pm 0{,}031$ $[-0{,}025;\ +0{,}097]$
agente v2, anche none, keep	$+0{,}059 \pm 0{,}037$ $[-0{,}009;\ +0{,}138]$	$+0{,}052 \pm 0{,}034$ $[-0{,}010;\ +0{,}123]$

A prevalenza reale la precision di `identity` scende da 0,873 a 0,468.

Il veto del giudice v2 la aumenta di circa 0,13 ($+0{,}127 \pm 0{,}045$) con una perdita di recall trascurabile ($-0{,}011 \pm 0{,}007$), e l'intervallo della differenza di *F1* di detection ($+0{,}101 \pm 0{,}034$) esclude lo zero; per l'agente completo gli intervalli comprendono lo zero.

4.3.5 Valutazione di conferma

La conferma usa `test_confirm` (nessuna frase in comune con `test_sample`) con le sole configurazioni del protocollo, `identity` e `judgev2` (le configurazioni che, dati i risultati delle metriche, sembrano le più interessanti da confrontare).

Il criterio, fissato in anticipo, richiede che il limite inferiore dell'intervallo al 95% della differenza di *F1* di detection riponderata sia maggiore di zero.

Le altre configurazioni sono state eseguite dopo, in modo esplorativo, al fine di avere una migliore visione complessiva dei risultati ottenuti valutando il sistema una seconda volta.

Tabella 8: `test_confirm`: 225 coppie (100 senza interazione, 125 con interazione), 184 frasi; oltre a `identity` e `judge v2`, le configurazioni sono esplorative.

Configurazione	micro F1	macro F1	det. F1	det. P / R	FP	persi
baseline	$0{,}757 \pm 0{,}035$	$0{,}701 \pm 0{,}072$	$0{,}879 \pm 0{,}022$	0,794 / 0,984	32	2
identity	$0{,}765 \pm 0{,}035$	$0{,}707 \pm 0{,}073$	$0{,}888 \pm 0{,}021$	0,809 / 0,984	29	2
judge v1 (veto)	$0{,}769 \pm 0{,}036$	$0{,}668 \pm 0{,}076$	$0{,}878 \pm 0{,}022$	0,862 / 0,896	18	13
agente v1, anche none, veto	$0{,}771 \pm 0{,}036$	$0{,}669 \pm 0{,}076$	$0{,}885 \pm 0{,}022$	0,847 / 0,928	21	9
agente v1, anche none, keep	$\mathbf{0{,}777 \pm 0{,}035}$	$\mathbf{0{,}732 \pm 0{,}080}$	$\mathbf{0{,}892 \pm 0{,}021}$	0,810 / 0,992	29	1
judge v2 (veto)	$0{,}773 \pm 0{,}035$	$0{,}712 \pm 0{,}073$	$0{,}892 \pm 0{,}021$	0,833 / 0,960	24	5
agente v2, anche none, veto	$0{,}767 \pm 0{,}034$	$0{,}694 \pm 0{,}067$	$0{,}882 \pm 0{,}021$	0,799 / 0,984	31	2
agente v2, anche none, keep	$0{,}773 \pm 0{,}034$	$0{,}699 \pm 0{,}067$	$0{,}887 \pm 0{,}021$	0,796 / 1,000	32	0

Il micro F1 sta tra 0,757 e 0,777 (errore standard di circa 0,035) e la macro F1 è poco informativa, perché `int` ha solo 5 coppie.

In questo test il giudice v1 elimina più falsi positivi (18 contro 29 di `identity`) ma perde 13 interazioni, mentre il v2 ne ha 24 e ne perde 5.

L'agente v2 non riduce i falsi positivi (31 e 32), replicando, come nel test precedente, che ciclo e controllo dei `none` non aggiungono nulla al veto.

Tabella 9: `test_confirm`: stima riponderata ± errore standard [IC 95%]; ultima riga: differenza appaiata (endpoint primario: *F1* di detection).

	detection F1	micro F1
identity	$0{,}582 \pm 0{,}036$ [0,52; 0,66]	$0{,}478 \pm 0{,}039$ [0,41; 0,56]
judge v2 (veto)	$0{,}615 \pm 0{,}040$ [0,54; 0,70]	$0{,}507 \pm 0{,}042$ [0,44; 0,60]
Differenza (v2 − identity)	$+0{,}033 \pm 0{,}021$ [−0,004; +0,079]	$+0{,}029 \pm 0{,}018$ [−0,002; +0,068]

Il criterio non è quindi raggiunto: la differenza di *F1* di detection riponderata è $+0{,}033 \pm 0{,}021$, con intervallo [−0,004; +0,079] (1961 ricampionamenti validi su 2000, perché con 5 coppie `int` a volte la classe manca).

Neppure il micro F1 riponderato ($+0{,}029 \pm 0{,}018$) né il confronto appaiato (180 contro 177 etichette giuste: 5 riparate, 2 rotte, $p = 0{,}45$) lo raggiungono.

L'andamento è lo stesso del primo campione, ma l'effetto è circa un terzo (+0,033 contro +0,101): è compatibile con una stima gonfiata dalla selezione (`judgev2` è stato scelto basandosi sul primo campione), senza poterla distinguere da una differenza reale.

Inoltre, le stime assolute non sono confrontabili tra i due campioni: il controllo di identità risolve 23 coppie su 100 senza interazione (7 nel `test_sample`).

In conclusione, la conferma non permette di affermare che il veto del giudice migliori il sistema: replica la direzione del primo risultato, che resta quindi un indizio esplorativo, mentre conferma, su dati nuovi, che il ciclo e il controllo dei `none` non aggiungono nulla al solo veto (analisi esplorativa).

4.4 Analisi degli errori

Per capire che cosa il giudice corregge e che cosa resta sbagliato, si sono classificati gli errori ricorrenti dei due campioni di test (Tabella 10).

Tabella 10: Errori ricorrenti e relativa evidenza.

Tipo di errore	Descrizione	Evidenza
Negazione letta come interazione	la frase dichiara l'assenza di interazione ("nessuna interazione è stata osservata...")	9 delle 14 coppie riparate da `judgev2` nei due test
Elenchi e classe-membro	due farmaci che interagiscono entrambi con un terzo, o un farmaco e la sua classe, attribuiti alla coppia	4 delle 9 coppie riparate nel `test_sample`
Confusione `int` / `effect`	interazioni generiche predette come `effect`: è un confine tra etichette che il giudice v2 non contesta	19 `int` su 40 predette `effect` nella baseline, 11 dalla stessa frase
Menzioni multiple	stessa frase e stessi nomi, ma la relazione è annotata su un'altra occorrenza	14 dei 24 falsi positivi rimasti dopo `judgev2` nella conferma
Controllo dei `none` poco affidabile	il giudice contesta `none` corretti	34 su 71 nel test, 27 su 48 nella conferma
Estrattore che cede alle critiche	cambia etichetta dopo il rifiuto, spesso verso `none`	11 casi su 18 (dev, rubrica v1), solo 2 utili

Il giudice corregge gli errori locali e semantici (negazioni, elenchi, classe-membro), mentre quelli che restano dipendono in larga parte dalle convenzioni di annotazione (menzioni multiple, confine tra `int` ed `effect`) e sfuggono a un controllo che legge la sola frase.

Questo spiega perché il giudice riduca i falsi positivi senza migliorare in modo netto il micro F1: il margine di intervento è limitato.

Anche il Corpus non è del tutto coerente: una frase che nega l'interazione è `none`, mentre una frase simile sul farmaco TOBI ("profili di eventi avversi simili") è `effect`.

5 Conclusioni e lavori futuri

In questa sezione si traggono conclusioni, basandosi sui risultati della sezione precedente, sulle performance del sistema e si propongono spunti per lavori futuri riguardanti il progetto sviluppato.

5.1 Risposte alle domande di ricerca

Facendo riferimento alle domande di ricerca definite in 1.1:

- **RQ1**: un agente con giudice e ciclo di critica non migliora, in modo dimostrabile, un singolo modello linguistico.

L'ipotesi principale non è confermata (204 → 210 etichette giuste su 260, $p = 0{,}18$), e il ciclo con il controllo dei `none` non aggiunge nulla al solo veto del giudice (211 contro 210).

- **RQ2**: un giudice prudente corregge in modo riproducibile i risultati negati e, sul primo campione di test e alla prevalenza reale, riduce i falsi positivi con una perdita di recall trascurabile (risultato replicato anche dal secondo test).

  Le istruzioni contano: con la rubrica v1 il giudice come veto peggiora il sistema, mentre la v2 perde molte meno interazioni. La confusione tra `int` ed `effect` resta irrisolta.

- **RQ3**: al fine di valutare correttamente il sistema, con campioni piccoli, servono confronti appaiati sulle stesse coppie, errori standard e intervalli ottenuti con il bootstrap per frasi.

  Inoltre sono utili una stima riponderata sulla prevalenza reale (analisi esplorativa) e una conferma su dati nuovi.

5.2 Limiti del progetto

La mancata conferma dell'ipotesi principale non prova in modo assoluto che l'agente non migliori il sistema: i risultati indicano che i dati disponibili non permettono di stabilire se il miglioramento osservato sia reale o dovuto alla variabilità del campione.

Questo esito può dipendere da più fattori, alcuni sostenuti dai dati della Sezione 4.3 e altri solo plausibili.

- Il primo fattore è il margine ridotto di miglioramento per un giudice che legge la sola frase: una parte degli errori dipende dalle convenzioni di annotazione (Sezione 4.4) e dal confine di valutazione tra le etichette `int` ed `effect`.
- Il secondo fattore è la potenza statistica limitata: con 14 coppie discordanti e la proporzione osservata ne servirebbero circa il doppio per ottenere $p < 0{,}05$. I limiti del piano gratuito (quote giornaliere) e i tempi di sviluppo hanno infatti imposto campioni piccoli (con soli 5 `int` nella conferma) e impedito ripetizioni.
- Il terzo fattore (plausibile) è derivante dall'appartenenza dei modelli alla stessa famiglia e tipo *lite*: il giudice accetta 37 delle 47 etichette di interazione sbagliate del test, comportamento compatibile con una tendenza a confermare la risposta dell'estrattore [8, 13]; tuttavia la prudenza del giudice v2 è anche voluta, perché la rubrica chiede di accettare in caso di dubbio.

Inoltre, un altro limite del progetto sviluppato è la valutazione di frasi del Corpus con i farmaci già individuati: per elaborare testi interi servirebbero un passo di riconoscimento dei nomi dei farmaci e una divisione in frasi.

5.3 Lavori futuri

Di seguito si propongono alcuni sviluppi naturali riguardanti sia la risoluzione dei problemi riscontrati che l'estensione del progetto:

- l'integrazione di un giudice più capace, o appartenente a una famiglia di modelli diversa, per verificare se la capacità è il fattore limitante;
- la gestione esplicita della negazione (per esempio con una regola e alcuni esempi negati nel prompt dell'estrattore), in quanto frasi come *"nessuna interazione è stata osservata"* vengono lette come interazioni (e il giudice le corregge soltanto a posteriori);
- il raggruppamento delle menzioni dello stesso farmaco (per esempio nome commerciale e generico) in un'unica nomenclatura tramite RxNorm (perché il Corpus annota la relazione solo su alcune occorrenze): valutare per farmaco, e non per menzione, eviterebbe di penalizzare predizioni corrette;
- il *fine-tuning* del modello usato dall'estrattore (per esempio un encoder biomedico o un LLM) sul *train* (circa 25.700 coppie);
- l'integrazione della valutazione su testi interi, come i foglietti illustrativi, includendo il riconoscimento dei farmaci (tecniche di *Named Entity Recognition*) e la divisione in frasi.

Riferimenti bibliografici

[1] Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. The hitchhiker's guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Melbourne, Australia, 2018. URL `https://aclanthology.org/P18-1128/`.

[2] Google. Gemini API: rate limits, 2026. URL `https://ai.google.dev/gemini-api/docs/rate-limits`. Ultimo accesso: 20 ottobre 2026.

[3] María Herrero-Zazo, Paloma Martínez, and Isabel Segura-Bedmar. Annotation issues in pharmacological texts. *Procedia – Social and Behavioral Sciences*, 95:211–219, 2013. doi: 10.1016/j.sbspro.2013.10.641.

[4] María Herrero-Zazo, Isabel Segura-Bedmar, Paloma Martínez, and Thierry Declerck. The DDI corpus: An annotated corpus with pharmacological substances and drug–drug interactions. *Journal of Biomedical Informatics*, 46(5):914–920, 2013. doi: 10.1016/j.jbi.2013.07.011.

[5] LangChain. LangGraph: low-level orchestration framework for building stateful agents, 2026. URL `https://github.com/langchain-ai/langgraph`. Ultimo accesso: 19 ottobre 2026.

[6] Aman Madaan, Niket Tandon, Prakhar Gupta, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. URL `https://arxiv.org/abs/2303.17651`.

[7] Stuart J. Nelson, Kelly Zeng, John Kilbourne, Tammy Powell, and Robin Moore. Normalized names for clinical drugs: RxNorm at 6 years. *Journal of the American Medical Informatics Association*, 18(4):441–448, 2011. doi: 10.1136/amiajnl-2011-000116.

[8] Arjun Panickssery, Samuel R. Bowman, and Shi Feng. LLM evaluators recognize and favor their own generations. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024. URL `https://arxiv.org/abs/2404.13076`.

[9] Isabel Segura-Bedmar. DDICorpus: repository del corpus (licenza cc by-nc), 2026. URL `https://github.com/isegura/DDICorpus`. Ultimo accesso: 19 ottobre 2026.

[10] Isabel Segura-Bedmar, Paloma Martínez, and María Herrero-Zazo. SemEval-2013 task 9: Extraction of drug-drug interactions from biomedical texts (DDIExtraction 2013). In *Second Joint Conference on Lexical and Computational Semantics (*SEM), Volume 2: Proceedings of the Seventh International Workshop on Semantic Evaluation (SemEval 2013)*, pages 341–350, Atlanta, Georgia, June 2013. Association for Computational Linguistics. URL `https://aclanthology.org/S13-2056/`.

[11] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. URL `https://arxiv.org/abs/2303.11366`.

[12] U.S. National Library of Medicine. RxNav e RxNorm, 2026. URL `https://lhncbc.nlm.nih.gov/RxNav/`. Ultimo accesso: 19 ottobre 2026.

[13] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, 2023. URL `https://arxiv.org/abs/2306.05685`.